GUÍA DE LABORATORIO

INFORMACIÓN BÁSICA					
ASIGNATURA:	INTRODUCCION AL DESARROLLO DE NUEVAS PLATAFORMAS (E)				
TÍTULO DE LA PRÁCTICA:	Persistencia de datos con Room Database en Jetpack Compose				
NÚMERO DE PRÁCTICA:	9	**AÑO LECTIVO:**	2026B	**NRO. SEMESTRE:**	VIII
TIPO DE PRÁCTICA:	**INDIVIDUAL**				
	GRUPAL	**X**	**MÁXIMO DE ESTUDIANTES**		2
FECHA INICIO:	27/10/2026	**FECHA FIN:**	31/10/2026	**DURACIÓN:**	100 minutos
RECURSOS A UTILIZAR: Android Studio (Quail o superior), Kotlin, Jetpack Compose, emulador Android o dispositivo físico (API 34 o superior)					
DOCENTE(s): Mg Roxana Evelyn Limache Calatayud					

OBJETIVOS/TEMAS Y COMPETENCIAS	
OBJETIVOS: - Comprender los componentes de Room (Entity, DAO, Database) y su relación con SQLite. - Diseñar un modelo de datos con entidades relacionadas para persistir información de los edificios del campus. - Implementar operaciones CRUD y una consulta relacional entre entidades mediante Room.	
TEMAS: - Room: Entity, DAO y Database - Relaciones uno-a-muchos con @Relation - Coroutines y Flow para operaciones asíncronas con Room	
COMPETENCIAS	C.a Aplica de forma flexible técnicas, métodos, principios, normas, estándares y herramientas de ingeniería necesarias para la construcción de software e implementación de sistemas de información.
	C.b Investiga nuevos modelos, metodologías, técnicas, herramientas y tecnologías por ser necesarias para mantener la vigencia en el desempeño profesional.
	C.c
	C.d

CONTENIDO DE LA GUÍA

I. MARCO CONCEPTUAL

Revisar el siguiente recurso antes de la sesión:

- Video: "Crea tu Primera App Android: CRUD, Base de Datos Room, Kotlin y Compose"
  https://www.youtube.com/watch?v=ueCxfv--2oU

- Android Developers, Room:
https://developer.android.com/training/data-storage/room

De los archivos de texto a una base de datos local

Hasta ahora, los datos de la aplicación (por ejemplo la lista de edificios del Laboratorio 08) se han guardado en archivos de texto plano o listas en memoria. A medida que el proyecto crece y aparecen relaciones entre datos (un edificio tiene varios ambientes, un ambiente tiene varias notas, es decir, anotaciones de texto sobre su estado o uso, como “el proyector no funciona” o “se reserva los martes”), conviene usar una base de datos local. Room es la librería que Android recomienda para trabajar con SQLite de forma segura y verificada en tiempo de compilación, sin escribir SQL a mano en la mayoría de los casos.

Los tres componentes de Room

Room se construye sobre tres piezas: @Entity, una clase que representa una tabla (cada instancia es una fila); @Dao, una interfaz que declara las operaciones sobre esa tabla (@Insert, @Update, @Delete, @Query); y @Database, una clase abstracta que reúne las entidades y expone los DAO, instanciada una sola vez con Room.databaseBuilder(...). La Figura 1 resume cómo se relacionan estos componentes con SQLite.

El Composable llama al DAO; Room traduce a SQL y persiste en SQLite



Figura 1. Los tres componentes de Room y su relación con SQLite (elaboración propia).

Relaciones uno-a-muchos

Cuando dos tablas están relacionadas (por ejemplo, un edificio que tiene varios ambientes), la tabla "muchos" guarda una clave foránea (por ejemplo edificioId) que referencia a la tabla "uno". Para consultar ambos niveles juntos, Room permite definir una clase intermedia con @Embedded para la entidad principal y @Relation para la lista de entidades relacionadas, como se resume en la Figura 2; una consulta @Transaction @Query(...) que devuelva esa clase intermedia trae el edificio junto con todos sus ambientes en una sola llamada.

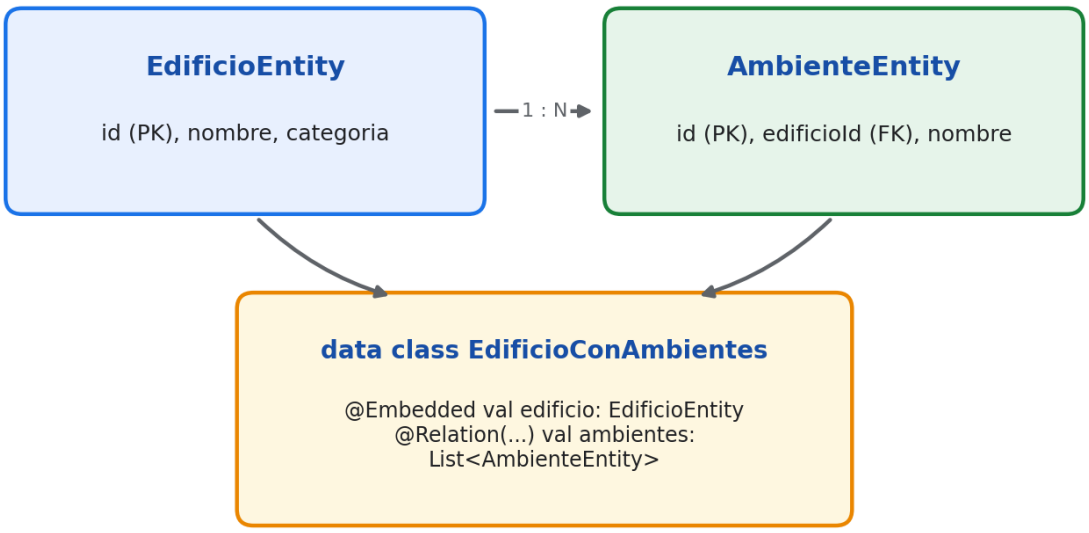


Figura 2. Relación uno-a-muchos entre EdificioEntity y AmbienteEntity mediante @Relation (elaboración propia).

II. EJERCICIO/PROBLEMA RESUELTO POR EL DOCENTE

A continuación se detallan, paso a paso, las instrucciones para persistir edificios y ambientes con Room, incluyendo una consulta relacional. También puede apoyarse en el siguiente video tutorial:

Video de referencia: "Crea tu Primera App Android: CRUD, Base de Datos Room, Kotlin y Compose" https://www.youtube.com/watch?v=ueCxfv--2oU

Paso 1. Crear un nuevo proyecto.

nombre sugerido para el proyecto: "PersistenciaRoom_Compose", Package name: com.example.persistenciaroom_compose, Language: Kotlin, Minimum SDK: el que venga por defecto en la plantilla (recuerden usar un emulador o dispositivo con API 34 o superior, según la sección de Recursos a utilizar).

Paso 2. Configurar Gradle para usar Room con KSP. Se modifican tres archivos, en este orden (las versiones indicadas funcionaron con Android Studio con AGP 9.3.3 y Kotlin 2.2.10; si tu proyecto usa otras versiones, revisa la nota que sigue a los códigos):

Código (gradle/libs.versions.toml – agregar las líneas de ksp):

```
[versions]
agp = "9.3.3"
kotlin = "2.2.10"
ksp = "2.3.6"
# ...el resto de tus versiones existentes

[plugins]
android-application = { id = "com.android.application", version.ref = "agp" }
kotlin-compose = { id = "org.jetbrains.kotlin.plugin.compose", version.ref = "kotlin" }
ksp = { id = "com.google.devtools.ksp", version.ref = "ksp" }
# ...el resto de tus plugins existentes
```

Código (build.gradle.kts del proyecto, el que dice "Project: PersistenciaRoom_Compose"):

```
plugins {
    alias(libs.plugins.android.application) apply false
```

```
    alias(libs.plugins.kotlin.compose) apply false
    alias(libs.plugins.ksp) apply false
}
```

Código (build.gradle.kts del módulo app):

```
plugins {
    alias(libs.plugins.android.application)
    alias(libs.plugins.kotlin.compose)
    alias(libs.plugins.ksp)   // plugin de KSP declarado en libs.versions.toml
    // ...el resto de tus plugins existentes
}

dependencies {
    // ...el resto de tus dependencias existentes (compose, activity, material3, etc.)

    // Necesaria para viewModel(factory = ...) en Compose
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.11.0")

    // Room: las tres líneas deben llevar la MISMA versión (2.7.0 o superior)
    implementation("androidx.room:room-runtime:2.7.2")
    implementation("androidx.room:room-ktx:2.7.2")
    ksp("androidx.room:room-compiler:2.7.2")
}
```

Nota (versiones y errores de sincronización). Las versiones dependen de tu Android Studio; en algunos casos hay que cambiarlas:

- Plugin [id: 'com.google.devtools.ksp'] was not found: KSP está sin versión. Debe declararse con versión en libs.versions.toml y aplicarse con alias(libs.plugins.ksp) dentro de plugins { }. La línea ksp("androidx.room:room-compiler:...") va dentro de dependencies { }.
- [ksp] IllegalStateException: unexpected jvm signature V: ocurre con Room 2.6.1 y un KSP nuevo. Usar Room 2.7.0 o superior en las tres líneas. Si Gradle no encuentra 2.7.2, probar 2.7.0 o 2.8.0.
- Con AGP 9 se necesita KSP 2.3.1 o superior (se recomienda 2.3.6). Si el Sync indica que KSP no es compatible con tu versión de Kotlin, cambiar la versión de KSP (por ejemplo 2.3.12). En proyectos con AGP 8.x, KSP usa el formato <Kotlin>-<KSP>, por ejemplo 2.2.10-2.0.2.
- Unresolved reference 'compose' o 'viewModel': falta lifecycle-viewmodel-compose; usar la misma versión que lifecycleRuntimeKtx del libs.versions.toml.
- Después de editar los archivos, pulsar Sync Now. Si quedan errores rojos en crearBaseDeDatos o edificioDao, ejecutar Build > Rebuild Project para que KSP genere las clases de Room.

Paso 3. Agregar, debajo del package y antes de la clase MainActivity, todas las importaciones que se usarán en este ejercicio (incluye Preview, porque la plantilla del proyecto conserva Greeting y GreetingPreview; sin ese import aparece Unresolved reference 'Preview'):

Código (importaciones):

```
import android.content.Context
import android.os.Bundle
import androidx.activity.ComponentActivity
import androidx.activity.compose.setContent
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.fillMaxSize
import androidx.compose.foundation.layout.padding
import androidx.compose.foundation.lazy.LazyColumn
import androidx.compose.foundation.lazy.items
import androidx.compose.material3.Surface
import androidx.compose.material3.Text
import androidx.compose.runtime.Composable
import androidx.compose.runtime.collectAsState
```

```
import androidx.compose.runtime.getValue
import androidx.compose.ui.Modifier
import androidx.compose.ui.text.font.FontWeight
import androidx.compose.ui.tooling.preview.Preview
import androidx.compose.ui.unit.dp
import androidx.compose.ui.unit.sp
import androidx.lifecycle.ViewModel
import androidx.lifecycle.ViewModelProvider
import androidx.lifecycle.viewModelScope
import androidx.lifecycle.viewmodel.compose.viewModel
import androidx.room.Dao
import androidx.room.Database
import androidx.room.Delete
import androidx.room.Embedded
import androidx.room.Entity
import androidx.room.ForeignKey
import androidx.room.Insert
import androidx.room.PrimaryKey
import androidx.room.Query
import androidx.room.Relation
import androidx.room.Room
import androidx.room.RoomDatabase
import androidx.room.Transaction
import androidx.room.Update
import com.example.persistenciaroom_compose.ui.theme.PersistenciaRoom_ComposeTheme
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.launch
```

Paso 4. Definir la entidad EdificioEntity:

Código (EdificioEntity):

```
@Entity
data class EdificioEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val nombre: String,
    val categoria: String
)
```

Paso 5. Definir la entidad AmbienteEntity, agregando una ForeignKey que la relacione con EdificioEntity:

Código (AmbienteEntity):

```
@Entity(
    foreignKeys = [
        ForeignKey(
            entity = EdificioEntity::class,
            parentColumns = ["id"],
            childColumns = ["edificioId"]
        )
    ]
)
data class AmbienteEntity(
    @PrimaryKey(autoGenerate = true) val id: Int = 0,
    val edificioId: Int,
    val nombre: String
)
```

Paso 6. Crear la interfaz EdificioDao con las operaciones básicas de inserción, actualización, borrado y una consulta simple. Por ahora dejamos un comentario donde irá la consulta relacional:

Código (EdificioDao – Paso 6):

```
@Dao
interface EdificioDao {
    @Insert
    suspend fun insertarEdificio(edificio: EdificioEntity): Long

    @Insert
    suspend fun insertarAmbiente(ambiente: AmbienteEntity)

    @Update
    suspend fun actualizarEdificio(edificio: EdificioEntity)

    @Delete
    suspend fun eliminarEdificio(edificio: EdificioEntity)

    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerTodos(): List<EdificioEntity>

    // la consulta relacional se agrega en el Paso 7
}
```

Paso 7. Definir la clase intermedia EdificioConAmbientes con @Embedded y @Relation, y agregar en el mismo Dao una consulta @Transaction que la devuelva. Con esto, EdificioDao queda completo:

Código (EdificioConAmbientes y EdificioDao completo):

```
data class EdificioConAmbientes(
    @Embedded val edificio: EdificioEntity,
    @Relation(
        parentColumn = "id",
        entityColumn = "edificioId"
    )
    val ambientes: List<AmbienteEntity>
)

@Dao
interface EdificioDao {
    @Insert
    suspend fun insertarEdificio(edificio: EdificioEntity): Long

    @Insert
    suspend fun insertarAmbiente(ambiente: AmbienteEntity)

    @Update
    suspend fun actualizarEdificio(edificio: EdificioEntity)

    @Delete
    suspend fun eliminarEdificio(edificio: EdificioEntity)

    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerTodos(): List<EdificioEntity>

    @Transaction
    @Query("SELECT * FROM EdificioEntity")
    suspend fun obtenerConAmbientes(): List<EdificioConAmbientes>
}
```

Paso 8. Crear la clase AppDatabase reuniendo ambas entidades y exponiendo el Dao, junto a una función que la instancie una sola vez con Room.databaseBuilder:

Código (AppDatabase):

```
@Database(entities = [EdificioEntity::class, AmbienteEntity::class], version = 1)
abstract class AppDatabase : RoomDatabase() {
    abstract fun edificioDao(): EdificioDao
}

fun crearBaseDeDatos(context: Context): AppDatabase {
    return Room.databaseBuilder(
        context,
        AppDatabase::class.java,
        "app.db"
    ).build()
}
```

Paso 9. Crear un EdificiosViewModel que, al iniciar, inserte (si la base está vacía) 2 edificios de muestra con sus ambientes mediante una coroutine, y exponga el resultado de obtenerConAmbientes() como un StateFlow observable. Como el ViewModel necesita el Dao en su constructor, se agrega también un Factory sencillo:

Código (EdificiosViewModel):

```
class EdificiosViewModel(private val dao: EdificioDao) : ViewModel() {

    private val _edificios = MutableStateFlow<List<EdificioConAmbientes>>(emptyList())
    val edificios: StateFlow<List<EdificioConAmbientes>> = _edificios

    init {
        viewModelScope.launch {
            if (dao.obtenerTodos().isEmpty()) {
                val id1 = dao.insertarEdificio(
                    EdificioEntity(nombre = "Biblioteca Central", categoria = "Servicios")
                )
                dao.insertarAmbiente(AmbienteEntity(edificioId = id1.toInt(), nombre = "Sala de
lectura"))
                dao.insertarAmbiente(AmbienteEntity(edificioId = id1.toInt(), nombre =
"Hemeroteca"))

                val id2 = dao.insertarEdificio(
                    EdificioEntity(nombre = "Facultad de Ingeniería", categoria = "Académico")
                )
                dao.insertarAmbiente(
                    AmbienteEntity(edificioId = id2.toInt(), nombre = "Laboratorio de cómputo")
                )
            }
            _edificios.value = dao.obtenerConAmbientes()
        }
    }
}

class EdificiosViewModelFactory(private val dao: EdificioDao) : ViewModelProvider.Factory {
    override fun <T : ViewModel> create(modelClass: Class<T>): T {
        @Suppress("UNCHECKED_CAST")
        return EdificiosViewModel(dao) as T
    }
}
```

Paso 10. Crear el Composable ListaEdificiosConAmbientes, que observe el StateFlow del ViewModel y muestre el resultado en una LazyColumn, agrupando visualmente los ambientes bajo el nombre de cada edificio:

Código (ListaEdificiosConAmbientes):

```
@Composable
fun ListaEdificiosConAmbientes(viewModel: EdificiosViewModel) {
    val edificios by viewModel.edificios.collectAsState()

    LazyColumn(modifier = Modifier.fillMaxSize().padding(16.dp)) {
        items(edificios, key = { it.edificio.id }) { item ->
            Column(modifier = Modifier.padding(vertical = 8.dp)) {
                Text(
                    text = item.edificio.nombre,
                    fontWeight = FontWeight.Bold,
                    fontSize = 18.sp
                )
                item.ambientes.forEach { ambiente ->
                    Text(text = "- ${ambiente.nombre}", fontSize = 14.sp)
                }
            }
        }
    }
}
```

Paso 11. Dentro de MainActivity, crear la base de datos, el Dao y el Factory, y mostrar ListaEdificiosConAmbientes dentro de setContent:

Código (MainActivity.kt completo):

```
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)

        val database = crearBaseDeDatos(applicationContext)
        val dao = database.edificioDao()
        val factory = EdificiosViewModelFactory(dao)

        setContent {
            PersistenciaRoom_ComposeTheme {
                val viewModel: EdificiosViewModel = viewModel(factory = factory)
                Surface(modifier = Modifier.fillMaxSize()) {
                    ListaEdificiosConAmbientes(viewModel)
                }
            }
        }
    }
}
```

Paso 12. Ejecutar la aplicación en un emulador y verificar en la LazyColumn que los edificios aparecen junto con sus ambientes; cerrar y volver a abrir la aplicación y confirmar que los datos siguen ahí (persisten en la base de datos local).

Paso 13. Verificar la base de datos: Room usa SQLite y guarda el archivo app.db dentro del dispositivo. Con la app en ejecución, abrir View > Tool Windows > App Inspection > Database Inspector y revisar las tablas EdificioEntity y AmbienteEntity (deben verse 2 edificios y 3 ambientes).

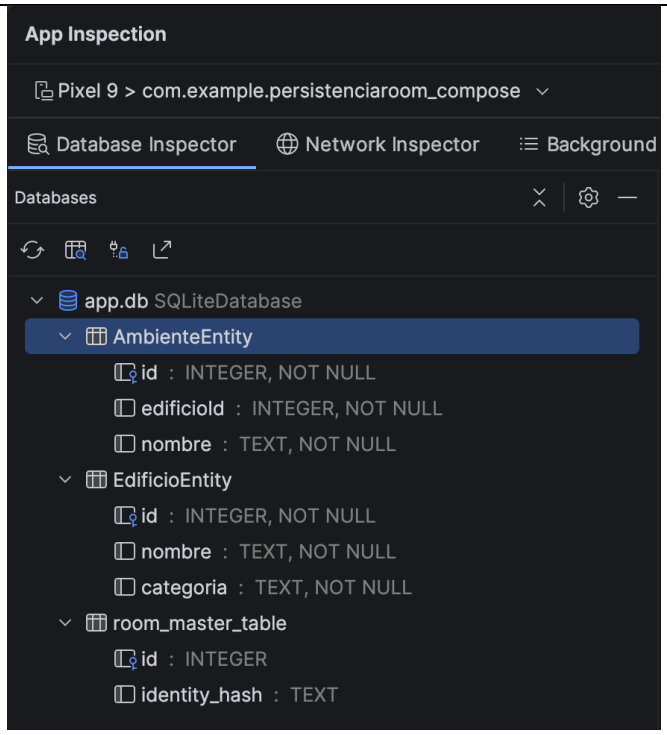


Figura 3. Tablas de la base de datos app.db (EdificioEntity, AmbienteEntity y room_master_table) vistas en el Database Inspector.

III. EJERCICIOS/PROBLEMAS PROPUESTOS

Amplía el modelo del ejercicio resuelto a al menos 3 tablas para almacenar la información de las edificaciones en una base de datos:

1. Diseña el diagrama del modelo de datos agregando una tercera entidad NotaEntity(id, ambienteId, texto) relacionada uno-a-muchos con AmbienteEntity (un ambiente puede tener varias notas).
2. Implementa el modelo completo con Room: las 3 entidades, sus claves foráneas, y el Dao correspondiente con @Insert, @Update y @Delete para las 3 tablas.
3. Realiza una segunda consulta relacional (por ejemplo, obtener un AmbienteEntity junto con su List<NotaEntity> mediante @Embedded/@Relation), distinta a la implementada en el ejercicio resuelto.
4. Agrega a la interfaz botones para eliminar y actualizar un edificio o ambiente existente, verificando que los cambios persistan.

Entregable: informe con el diagrama de las 3 tablas, y capturas de las dos consultas relacionales funcionando.

IV. CUESTIONARIO

1. ¿Lograste resolver completamente el Ejercicio/Problema Resuelto por el Docente (datos de edificios y ambientes almacenados con Room, que persisten al cerrar y volver a abrir la app)? Adjunta una captura de pantalla del emulador mostrando el resultado como evidencia.
2. ¿Qué otra alternativa de almacenamiento (además de Room) se podría emplear en una aplicación con necesidades similares? Justifica tu respuesta comparando ventajas y desventajas frente a Room.
3. ¿En dónde almacena Room los datos de la aplicación (almacenamiento interno o externo del dispositivo)? Justifica tu respuesta y explica qué implica para la privacidad de los datos del usuario.

V. REFERENCIAS Y BIBLIOGRÁFIA RECOMENDADAS:

[1] Android Developers, "Save data in a local database using Room". [En línea]. Disponible: https://developer.android.com/training/data-storage/room

[2] "Crea tu Primera App Android: CRUD, Base de Datos Room, Kotlin y Compose", video. [En línea]. Disponible: https://www.youtube.com/watch?v=ueCxfv--2oU

TÉCNICAS E INSTRUMENTOS DE EVALUACIÓN

TÉCNICAS:	**INSTRUMENTOS:**
Ejercicios propuestos	Rúbrica de evaluación

CRITERIOS DE EVALUACIÓN

- Cumplimiento de los objetivos y ejercicios propuestos en la guía.
- Correcta aplicación de los conceptos y herramientas del tema desarrollado.
- Funcionamiento correcto de lo implementado (compilación/ejecución sin errores).
- Informe (formato estudiante) completo, con evidencias y cuestionario resuelto.
- Entrega dentro del plazo establecido.
- Se permite el uso de Inteligencia Artificial (IA) como fuente de consulta o apoyo para resolver los ejercicios; sin embargo, el desarrollo completo del laboratorio no puede basarse únicamente en respuestas generadas por IA. El estudiante debe evidenciar comprensión propia mediante la justificación razonada de sus respuestas, especialmente en el cuestionario.